

SVM-LO: An Accurate, Robust, Real-time LiDAR Odometry with Segmentation Voxel Map for Autonomous Vehicles

Xiaolong Cheng¹, Tianxiao Ma¹, Keke Geng^{*1}, Zhichao Liu¹, Ziwei Wang¹ and Guodong Yin¹,
Senior Member, IEEE

Abstract— Self-localization is an essential technology for autonomous vehicles. This work proposed a simple and effective LiDAR odometry based on the segmentation voxel map termed SVM-LO. Unlike other LiDAR odometry, the SVM-LO is unlimited by LiDAR scanning modes and environment changes. Our approach consists of two components: the front-end, which acquires key points and the local map, and the back-end, which obtains the robust poses by adaptive mix iterative closest point (AM-ICP). We design an adaptive voxel classifier for extracting more stable key points and constructing the segmentation voxel map. To guarantee system real-time performance, we propose the adaptive dual-subsampling module and optimized the store strategy of the local map. We conduct extensive experiments comparing SVM-LO with state-of-the-art technology on different datasets, the results demonstrate that the SVM-LO is generic, robust, and accurate. Our system is able to run in real-time under multi-scale scenarios, narrow scenarios, and degradation scenarios. SVM-LO eschews reliance on LiDAR scanning characteristics and other sensors, thus it can be widely used in a variety of autonomous driving scenarios.

I. INTRODUCTION

LiDAR is regarded as a fundamental sensor for autonomous vehicles due to its ability to acquire environment depth information and high measurement accuracy (e.g., self-localization [1], 3D object detection [2], road boundary detection [3], etc.). Simultaneous localization and mapping (SLAM) technology has long been a popular research problem, which is an important technology in autonomous navigation for driverless vehicles. LiDAR SLAM has been developed over decades and has many mature algorithms [4-7]. However, most of these algorithms have a huge number of configuration parameters and are only suitable for specific scenarios, which does not apply to autonomous driving. Meanwhile, the emergence of new solid-state LiDAR in recent years has rendered many odometry frameworks unsuitable, due to its particular scanning mode [8].

In this paper, we return to the roots of LiDAR odometry, using the fixed nature of the point cloud to design a widely applicable odometry framework. The voxel grid is a common storing method for point cloud, due to it is simple and fast. Many methods [9, 10] use voxel to downsample point clouds, but voxel size affects the number of sampling points and data integrity. For the normal distribution transform (NDT) [11]

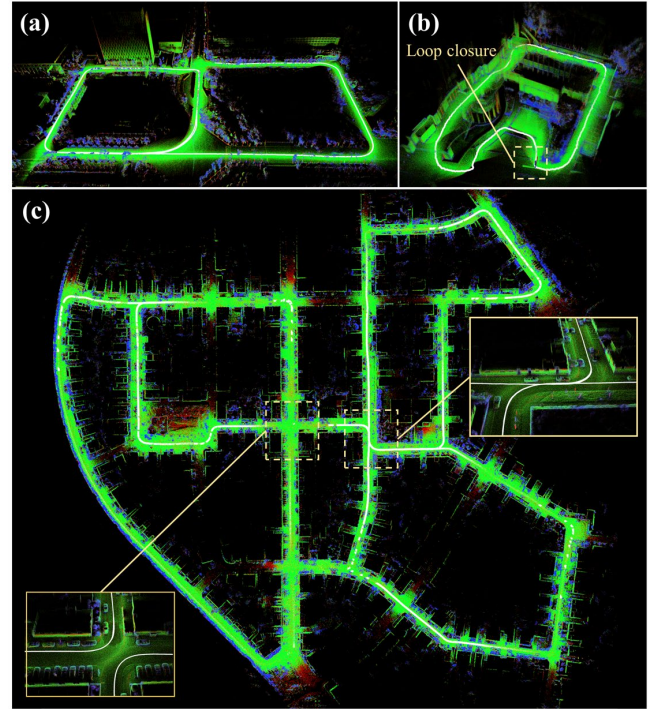


Fig. 1: The segmentation voxel map of SVM-LO. (a) The sequence “SEU_04” of the SEU dataset is collected by the intelligent vehicle on the SEU campus. (b) The sequence “hku campus seq 03” of the R3LIVE-dataset is collected by a handheld device on the HKU campus. (c) The sequence “kitti 00” of the KITTI dataset is collected by the vehicle in the city.

algorithm, the voxel size directly affects the alignment result. Choosing an appropriate voxel size has always been a challenge in LiDAR odometry.

To solve the above problems, we propose an odometry framework based on the segmentation voxel map. We first analyze the spatial characteristics of the per-frame point cloud using the initial voxel size, then the voxel size is resized according to the spatial factor for point cloud segmentation and subsampling. In addition, different types of key points use different residual equations for iterative optimization. These strategies allow our system to operate accurately and stably in different scenarios without adjusting the system parameters.

We obtain some result maps as shown in Fig. 1, where we can tell the system rationality and accuracy from the point

This work was supported in part by the National Natural Science Foundation under Grant no. 52272414 and 51905095, and Jiangsu Graduate Innovative Research Program under Grant no. SJCX24_0072. (Corresponding author: Keke Geng.)

¹Xiaolong Cheng, Tianxiao Ma, Keke Geng, Zhichao Liu, Ziwei Wang, Guodong Yin are with the Department of Mechanical Engineering, Southeast University, Nanjing, 201804, China (email: {230238059, 230230347, jsgengke, 230248037, 230238070, ygd}@seu.edu.cn).

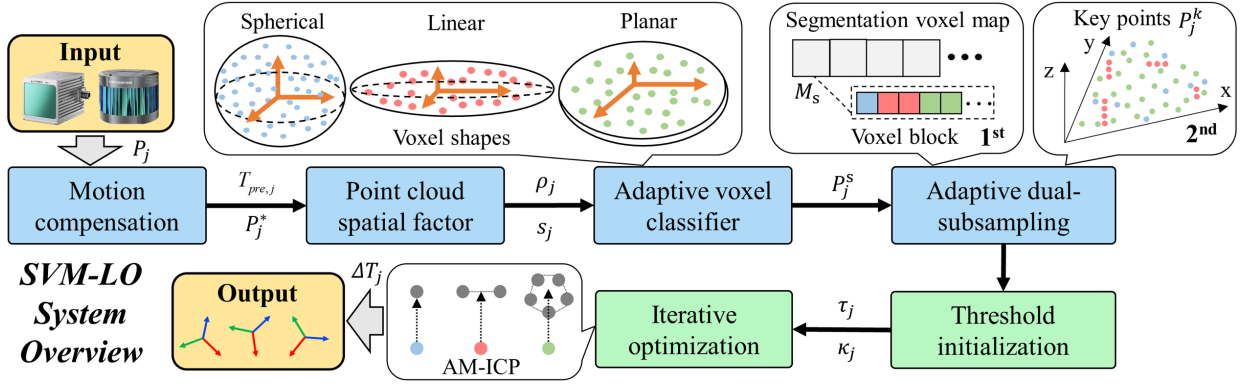


Fig. 2: This figure shows the overview of SVM-LO. The yellow blocks represent the input and output of the system. The blue blocks are the data processing front end and the green blocks are the iterative optimization back end.

cloud segmentation results. In summary, our work follows key contributions.

- 1) We propose a novel LiDAR odometry framework termed SVM-LO, which applies to different LiDAR scanning modes and scale scenes.
- 2) Propose an adaptive voxel classifier to segment the point cloud. It is fast, adaptive, and simple, and only uses the covariance matrix to get good point cloud segmentation results.
- 3) The novel method of using adaptive dual-subsampling to obtain the key points and the segmentation voxel map.
- 4) We use adaptive mix iterative closest point (AM-ICP) to robust iterative optimization, which enables stable operation in narrow and degeneration scenarios.
- 5) We have conducted extensive experiments on the different datasets to validate the universality, robustness, and accuracy of the SVM-LO.

The rest of this paper is organized as follows. In Sec. 2, we discuss the point cloud registration method and LiDAR odometry. Sec. 3 introduces our system framework. Extensive experiments in Sec. 4. Sec. 5 concludes this work.

II. RELATED WORK

In this chapter, we review existing point cloud registration methods and LiDAR odometry. Analyzing the challenges of LiDAR for autonomous localization.

A. LiDAR Odometry

The main goal of LiDAR odometry is to obtain the relative pose between two point clouds. Point-to-point matching [12] is a general and simple approach for LiDAR registration. However, the point-to-point is sensitive to noise and outlier points. Censi proposes the point-to-line iterative closest point to enhance robustness and the point-plane ICP [13] is proposed for accelerated convergence. Serafin et al [14] provide a method for establishing the pose transformation between two clouds using the point cloud 3D structure. Dellenbach et al propose a novel approach CT-ICP [7], which uses a sparse voxel structure to store point clouds for real-time processing. The KISS-ICP [9] uses voxel downsampling and adaptive thresholds to improve ICP iteration speed and accuracy.

However, all ICP-based methods require a high initial pose and have low calculation speed in the large-scale point cloud. The Normal Distributions Transform (NDT) [11] transforms the point cloud into multi-dimensional vectors with normal distribution for alignment, which is not affected by small local changes and is suitable for large point clouds. The scholars combined ICP and NDT [15], which use NDT for coarse matching and then use ICP for fine matching. Moreover, to avoid using the full point cloud for alignment, Zhang et al [4] propose a method for extracting edge points and planar points for alignment, which makes the real-time 3D SLAM possible. The LeGo_LOAM [5] further improves the algorithm speed and accuracy by ground segmentation of the point cloud. Pan et al [16] use five types of features (e.g., ground, façade, columns, etc.), and propose the multi-scale quantitative linear least squares iterative nearest point algorithm to match the features. However, the above approaches require modifying system parameters based on the environment and LiDAR type.

B. Dynamic Structure in Local Map

The autonomous vehicle system has high requirements for robustness, real-time, and stability of algorithms. This requires the local map structure that can be updated in real time and quickly k NN search. Octree [17] and k-d tree [18] are well-know tree data structures that deal with multidimensional spaces, the k-d tree is widely used in LiDAR odometry [19] because of its balance structure and efficient k NN algorithm. However, updating and deleting the k-d tree structure each scan makes system inefficient. The ikd-Tree [10] efficiently deletes and inserts new nodes for faster point cloud alignment. Yuan et al [20] build and update map efficiently with a new voxel structure organized by hash table and octree. The KISS-ICP [9] use hash table to store voxel blocks, which limit the local map size by the maximum storage number and distance range. However, the constant maximum storage number is not appropriate in multi-scale scenarios. In this work, we optimize storage strategies and adaptive thresholds to manage local map to meet real-time requirement.

III. PROPOSED SYSTEM

A. System Overview

The notations in this paper are summarized in TABLE I. Our system overview is shown in Fig. 2, which consists of *the data processing front end and the iterative optimization back end*.

TABLE I: NOMENCLATURE

Notation	Explanation
P_j	The j th frame point cloud.
P_j^*	The de-skewed point cloud of j th frame.
P_j^s	The segmentation point cloud of j th frame.
P_j^k	The key points of j th frame.
ρ_j	The average point number of one voxel in j th frame.
V_j	The average occupied volume of one point in j th frame.
s_j	The spatial factor of the j th frame.
v	The initial voxel size.
δ	The judgment threshold of the adaptive voxel classifier.
v_{map}	The voxel size of the local map.
v_j^1	The first downsampling voxel size of the j th frame.
v_j^2	The second downsampling voxel size of the j th frame.
M_s	The local segmentation voxel map.
N_{max}^j	The max point number for one voxel block.
τ_j	The search distance of j th frame.
κ_j	The scale parameter of the confidence detector.
σ	The iterative optimization threshold.
$T_{pre,j}$	The initial prediction pose of the j th frame.
ΔT_{am_icp}	The correction pose of the j th frame.
T_j	The final pose of the j th frame.

The motion compensation of input point cloud P_j is based on the constant velocity model, which can obtain the de-skewed point cloud P_j^* and initial pose $T_{pre,j}$. Compute the spatial factor s_j , for adaptive voxel classifier to obtain the segmentation point cloud P_j^s . Then, the key points P_j^k and segmentation voxel map M_s can be obtained by the adaptive dual-subsampling module. Finally, the robust iterative optimization with thresholds, τ_j and κ_j , is used to obtain the position correction ΔT_{am_icp} .

B. Motion Compensation

LiDAR generally puts points acquired at different moments into the same coordinate frame, which renders the deviation between the point cloud and the actual environment. Therefore it is essential to motion compensation of the point cloud before alignment. To not add additional sensors, we use the constant velocity model like KISS-ICP [9] to distort the point cloud. This method is simple and general, and the process of point cloud distortion correction is as follows.

The constant velocity model utilizes previous pose estimates $T_{j-1} = (R_{t-1}, t_{t-1})$, $T_{j-2} = (R_{t-2}, t_{t-2})$ and time intervals Δt to estimate the relative pose $T_{pre,j} = (R_t, t_t)$, translation

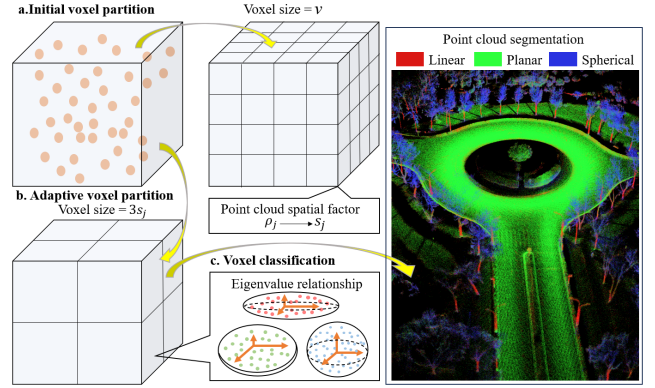


Fig. 3: The pipeline of the adaptive voxel classifier.

velocity v_t , and angular velocity ω_t at the current moment t are obtained:

$$T_{pred,j} = \begin{bmatrix} R_{t-2}^T R_{t-1} & R_{t-2}^T (t_{t-1} - t_{t-2}) \\ 0 & 1 \end{bmatrix}, \quad (1)$$

$$v_t = \frac{R_{t-2}^T (t_{t-1} - t_{t-2})}{\Delta t},$$

$$\omega_t = \frac{\text{Log}(R_{t-2}^T R_{t-1})}{\Delta t},$$

where $\text{Log}: SO(3) \rightarrow \mathfrak{R}^3$ extracts the axis-angle representation. The LiDAR gets scanned in to obtain a point cloud P_j . Each point $p_i \in P_j$ has a relative timestamp $t_i \in [0, \Delta t]$. So the corrected point p_i^* of each point p_i can be obtained by the translation velocity v_t and rotation velocity ω_t at the current time t , thus obtaining the de-distorted point cloud P_j^* :

$$p_i^* = \text{Exp}(t_i \omega_t) p_i + t_i v_t, \quad (2)$$

where $\text{Exp}: \mathfrak{R}^3 \rightarrow SO(3)$ computes a rotation matrix from an axis-angle representation.

C. Adaptive Voxel Classifier

Point cloud segmentation can provide more information and constraints for the pose iterative optimization [12], thus improving the effectiveness of the alignment and enhancing the system robustness. Deep learning-based point cloud segmentation methods [21] have high GPU requirements and limited application scenarios, which makes the odometry system complex and heavy. The voxel shapes can be classified by the relationship between the eigenvalues [11], point cloud segmentation is possible through shapes. This classification method is simple, but the classification results are highly sensitive to the voxel size.

For the above issues, we propose the adaptive voxel classifier, which can adaptively adjust the voxel size according to the environment to get good point cloud segmentation results. The pipeline of this method is shown in Fig. 3, which has three steps, initial voxel partition, adaptive voxel partition, and voxel classification.

First, divide the point cloud P_j^* with the initial voxel size v , we set v to 1m. Afterward, all voxels containing points are added to obtain the point cloud volume, and the average point number ρ_j of one voxel is easily calculated. The average occupied volume V_j of one point is

$$V_j = \frac{v^3}{\rho_j}. \quad (3)$$

We define the point cloud spatial factor as

$$s_j = \sqrt[3]{V_j}. \quad (4)$$

The spatial factor s_j can reflect the point cloud density and covering volume side length.

To classify voxels accurately, it is necessary to ensure that the number of points within a voxel is not too few or too many. So we set the voxel size to $3s_j$ for adaptive voxel partition and removed voxels that are less than 4 points. This handling ensures that the number of points within each voxel is around 27.

Then, we calculate the covariance matrix and eigenvalues for each voxel. For the point cloud P_v in one voxel, the point number is n , the mean value of each dimension that can be obtained

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n p_i^x, \quad \bar{y} = \frac{1}{n} \sum_{i=1}^n p_i^y, \quad \bar{z} = \frac{1}{n} \sum_{i=1}^n p_i^z, \quad (5)$$

where for each point $p_i \in P_v$. The covariance matrix C can be expressed as

$$C = \begin{bmatrix} C_{xx} & C_{xy} & C_{xz} \\ C_{yx} & C_{yy} & C_{yz} \\ C_{zx} & C_{zy} & C_{zz} \end{bmatrix}, \quad (6)$$

where C_{ab} denote the covariance between dimension $a \in (x, y, z)$ and dimension $b \in (x, y, z)$, which is calculated by equation

$$C_{ab} = \frac{1}{n} \sum_{i=1}^n (p_i^a - \bar{a})(p_i^b - \bar{b}). \quad (7)$$

The eigenvalues of the covariance matrix are calculated and sorted in decreasing order as $\lambda_1, \lambda_2, \lambda_3$. The voxel shape can be classified according to the determining conditions in Eq. (8), where δ is the judgment threshold.

$$\begin{cases} \text{Linear} & \lambda_1 > \delta\lambda_2, \lambda_1 > \delta\lambda_3 \\ \text{Planar} & \lambda_1 > \delta\lambda_3, \lambda_2 > \delta\lambda_3 \\ \text{Spherical} & \text{else} \end{cases} \quad (8)$$

D. Adaptive Dual-subsampling

Extracting the key points for registration is a common method in LiDAR odometry, as it ensures the real-time performance of systems. Using a fixed subsampling voxel size may render too few sampling points in small scenes and too many in large scenes. Hence, we propose the adaptive dual-subsampling, the first subsampling voxel size v_j^1 to

create a segmentation voxel map, and the second voxel size v_j^2 to obtain the key points P_j^k . Meanwhile, we consider that linear and planar features are more identifiable, and spherical features are mostly clustered points. Thus we pay more attention to the linear and planar features, the final downsampling size can be expressed as

$$v_j^1 = \alpha s_j \begin{cases} \alpha = 0.5\alpha & \text{else} \\ \alpha = 1.5\alpha & \text{spherical} \end{cases}, v_j^2 = \beta s_j \begin{cases} \beta = 0.5\beta & \text{else} \\ \beta = 1.5\beta & \text{spherical} \end{cases}, \quad (9)$$

where the α and β represent the expansion factor, we set α is 1.5 and β is 3.

The scan-to-map registration is more robust than the scan-to-scan alignment [22]. We use the voxel block with a fixed voxel size v_{map} to store the segmentation voxel map and limit the maximum point number N_{max}^j of per voxel block. In order to make the alignment more accurate and to improve computational efficiency, the N_{max}^j should be dynamically adjusted according to the scene characteristics

$$N_{max}^j = \frac{\rho_j}{\alpha^3} + N_{min}, \quad (10)$$

where N_{min} denotes the minimum storage number, ρ_j denotes the average point number of one voxel and α represents the expansion factor. This design makes the system more efficient and robust.

E. AM-ICP

Iterative closest point (ICP) is a classical method for point cloud alignment, where the search distance τ_j affects the iteration speed. Large environments require larger search ranges, while small environments require smaller search ranges. Thus, we design the search boundary τ_j changing with the sampling distance of the key points v_j^2 , expressed as Eq.(11), where β represents the expansion factor.

$$\tau_j = \beta v_j^2, \quad (11)$$

Through the search distance τ_j , the voxel search boundary η can be expressed as

$$\eta = \text{ceil}((\frac{2\tau_j}{\sqrt{3}} - v_{map}) / 2), \quad (12)$$

where $\text{ceil}(x)$ represents taking an integer to x . When the η greater than 0, it is not enough to find the target point only in the current voxel and the voxel search boundary needs to be expanded η voxels from the current voxel.

Adaptive mix iterative closest point (AM-ICP) is proposed to solve the problem that point-to-point alignment is sensitive to noise and unable to handle the variation of local shapes. The key points $P_j^k (j \in [1, n])$ are first transformed to the global coordinate frame based on the predicted pose $T_{pre,j}$, expressed as Eq. (13).

$$S_j = \{s_i = T_{j-1} T_{pre,j} p_i \mid p_i \in P_j^k\}. \quad (13)$$

Based on the classification results, the source points $S_j (j \in [1, n])$ can be classified into spherical points $P_s \in S_j$, linear points $P_l \in S_j$ and planar points $P_p \in S_j$. For each spherical point $p_i^s \in P_s (i \in [1, n_1])$, finding the nearest neighbour point q_1 in the local map M_s , the point-to-point distance residual r_{ps} is

$$r_{ps}^i = \|p_i^s - q_1\|. \quad (14)$$

For each linear point $p_j^l \in P_l (j \in [1, n_2])$, finding the two nearest points $q_1, q_2 \in M_s$ and calculating the point-line distance residual r_{pl}

$$r_{pl}^j = \frac{|(p_j^l - q_1) \times (q_2 - q_1)|}{\|q_2 - q_1\|}. \quad (15)$$

For each planar point $p_k^p \in P_p (k \in [1, n_3])$, finding the three nearest points $q_1, q_2, q_3 \in M_s$ and calculating the point-plane distance residual r_{pp} , see Eq. (16). The total residual r_{total} for AM-ICP is denoted as Eq. (17), where w is confidence weight. During the AM-ICP iteration process, the minimum change threshold σ is used as the convergence condition.

$$r_{pp}^k = \frac{|(p_k^p - q_1) \cdot ((q_2 - q_1) \times (q_3 - q_1))|}{\|(q_2 - q_1) \times (q_3 - q_1)\|}, \quad (16)$$

$$r_{total} = \sum_{i=1}^{n_1} (w_i \cdot \|r_{ps}^i\|^2) + \sum_{j=1}^{n_2} (w_j \cdot \|r_{pl}^j\|^2) + \sum_{k=1}^{n_3} (w_k \cdot \|r_{pp}^k\|^2), \quad (17)$$

$$n = n_1 + n_2 + n_3,$$

Calculating the confidence level for each point alignment can reduce the influence of dynamic objects and noise [23], the confidence detector is expressed as

$$w = \left(\frac{r}{\kappa_j + r} \right)^2, \quad (18)$$

where $w \in [0, 1]$ denotes confidence level, r denotes the point residual, κ_j is the scale parameter and the calculation equation is as follows

$$\kappa_j = \frac{v_{map}}{\sqrt[3]{N_{max}^j}}. \quad (19)$$

Iterate the above optimization process until convergence, the goal of optimization is the minimize total residual r_{total} , and the modified pose ΔT_{am_icp} can be obtained. The current pose T_j is expressed as

$$T_j = \Delta T_{am_icp} T_{pre,j} T_{j-1}. \quad (20)$$

IV. EXPERIMENTAL EVALUATION

In this section, we evaluate our method in three folds: **1.** To verify the impact of the adaptive module and AM-ICP on system performance, we do ablation experiments (Sec. B). **2.** To validate the localization accuracy of our system, we compare our system with state-of-the-art SLAM method on



Fig. 4: The collection platform of SEU-dataset is the intelligent vehicle equipped with AVIA LIDAR and GPS.

our private dataset and public dataset (Sec. C). **3.** To demonstrate the simple and computing efficiency of our approach, we analyze the system run time (Sec. D).

A. Datasets and Experimental Setup

1. Experimental Setup

To prove the generality of our method, we test it on the LiDAR datasets with different scanning modes. For solid-state LiDAR, we compare it with methods that perform well on solid-state LiDAR, including KISS-ICP [9], Livox-mapping, Loam_livox [8], FAST-LIO2 [10], Point-LIO [24]. Similarly, for mechanical LiDAR, we compare it with methods, including KISS-ICP [9], LOAM [4], LeGo_Loam [5], SUMA [22], SUMA++ [12], LiTAMIN2 [25], MULS [16].

For the sake of fair comparison, all experiments in this paper are conducted with the same parameter settings. We set the initial voxel size v and the local map voxel size v_{map} to 1m, the ICP convergence criterion σ is 10^{-4} , and the judgment threshold δ of the adaptive voxel classifier is 5.

II. Our Private Dataset: SEU-dataset

Our data collection platform is shown in Fig. 4. We use the intelligent vehicle to collect eight sequences including closed-loop scenarios, and scale-varying scenarios on the Southeast University campus. We use the AVIA LiDAR to collect point clouds and use the GPS data as the ground truth.

III. Public Dataset: KITTI-dataset

The KITTI dataset [26] is a popular autonomous driving benchmark. KITTI odometry dataset provides 11 sequences (00–10) with ground truth trajectories, including city, country, and highway. The point cloud is collected by the Velodyne HDL64E LiDAR.

B. Experiment-1: Ablation Study

The main contribution of our work consists of three parts: adaptive voxel classifier, adaptive dual-subsampling, and AM-ICP. The adaptive voxel classifier is a prerequisite for AM-ICP, so we test these two components together. For the validity and necessity of each module, we test ablation studies on the SEU_05 sequence which is the underground narrow

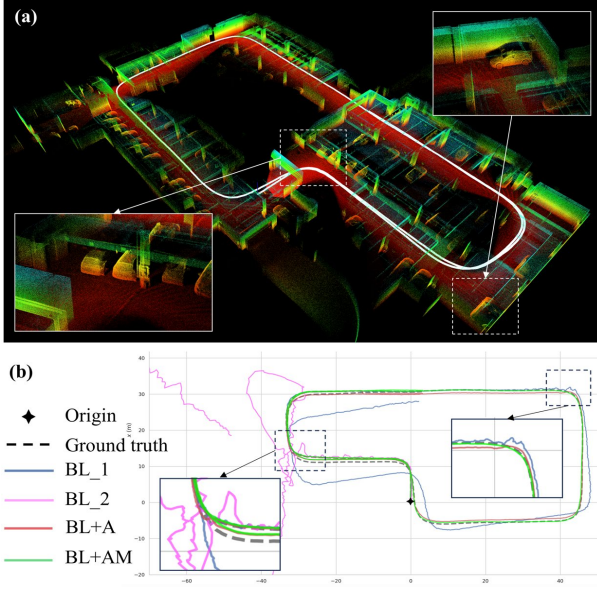


Fig. 5: (a) The 3D point cloud map build by SVM-LO. (b) The trajectories of ablation experiment.

TABLE II: THE APE/RPE OF OUR METHOD PIPELINE ON OUR DATASET. THE -- REPRESENTS THE SYSTEM FAILURE IN THE TEST.

Method	APE(m)			RPE(m)		
	Mean	Rmse	Std	Mean	Rmse	Std
BL_1	2.97	4.52	3.40	0.07	0.09	0.05
BL_2	--	--	--	--	--	--
BL+A	0.77	0.88	0.43	0.03	0.04	0.03
BL+AM	0.61	0.68	0.30	0.02	0.02	0.01

scene. We set two different initial downsampling sizes for illustrating the effect of voxel size on system performance, 1m (BL_1) and 2m (BL_2). Baseline plus adaptive dual-subsampling (BL+A) for validating the effectiveness of adaptive subsampling. Plus adaptive dual-subsampling and AM-ICP (BL+AM) demonstrate the contribution of AM-ICP.

The results are shown in TABLE II. We discover that using a fixed subsampling size is not applicable in this scene, the BL_2 failure is due to its large subsample size. The adaptive dual-subsampling solves this problem, which can adaptively adjust the subsampling size according to environmental changes. From Fig. 5, the trajectory obtained by using AM-ICP is smoother. Compared with simple point-to-point matching, AM-ICP can adjust the matching strategy according to the environmental feature, with higher accuracy and stability. The prerequisite for using AM-ICP is good voxel segmentation results. The adaptive voxel classifier accomplishes this point, and some classification results are shown in Fig.6. The categorization of environmental elements can be refined as follows: tree trunks, electrical poles, and analogous structures are classified as linear, vehicles, bushes, and plants are designated as spherical. Additionally, walls and floors are categorized as planar.

C. Experiment-2: Evaluation of Localization Accuracy

In this experiment, we use state-of-the-art SLAM methods to do comparison experiments on SEU and KITTI datasets to

verify the stability and localization accuracy of SVM-LO. The results are shown in TABLE III and TABLE IV, and the trajectories comparison on SEU-dataset are shown in Fig. 7.

For SEU-dataset, the sequences SEU_05 and SEU_08 are scale-varying scenarios, the sequences SEU_04, SEU_06, and SEU_07 are closed-loop scenarios, and the rest are normal driving scenarios. LiDAR algorithms (KISS-ICP, Livox-mapping, Loam_livox) appear to fail in varying degrees, because of using fixed parameters for downsampling or extracting features. LiDAR-Inertial systems (FAST-LIO2, Point-LIO) can stably run in all sequences. But without the closed-loop module, the system drifts too much when running for a long time. The total distance in our dataset is 13.151 KM and the average absolute position error of SVM-LO is 1.427m.

Our dataset is collected on campus with few traffic targets and slow driving speeds. To enhance the validation of the robustness and stability of our algorithm, we test on the popular KITTI dataset with many LiDAR systems. From the TABLE III, the performance of our algorithms is competitive and the mean average translation error (ATE) is 0.47%.

The algorithm is proved to be highly robust and accurate through the above experiments, some result maps are shown in Fig. 1. Meanwhile, our algorithms apply to handheld scenarios as well as self-driving scenarios. We use the R3LIVE dataset [27] to build the HKU campus map and the starting point and the endpoint realize a loop closure, shown in Fig.1(b).

D. Experiment-3: Run Time Analysis

In this section, we analyze the run time of our system to verify the real-time quality. We test the average frame processing time of SVM-LO on the CPU-only laptop (featuring an Intel R7-5800H CPU and 16GB RAM). The average processing time per frame is 22.46 ms on SEU-dataset and 54.38 ms on KITTI-dataset. The typical LiDAR frequency is 10 Hz, therefore the SVM-LO system meets the real-time requirement.

V. CONCLUSION

In this paper, we propose an accurate, robust, and real-time LiDAR odometry termed SVM-LO. The system pipeline is based on the segmentation voxel map which allows real-time adjustment of voxel segmentation size and downsampling size in response to environment changes. In addition, we propose AM-ICP to improve system robustness and location accuracy. We evaluated our method and compared it with the state-of-the-art algorithms available on our dataset and public dataset, and the results suggest that the performance of our method is comparable with LiDAR-Inertial tightly coupled SLAM. After extensive tests, our system supports LiDAR with different scanning modes and can operate stably in real-time under multi-scale scenarios, point cloud degradation scenarios, handheld scenarios, and autonomous driving scenarios. In the future, we will use the point cloud depth image to aid point cloud segmentation and dynamic object removal.

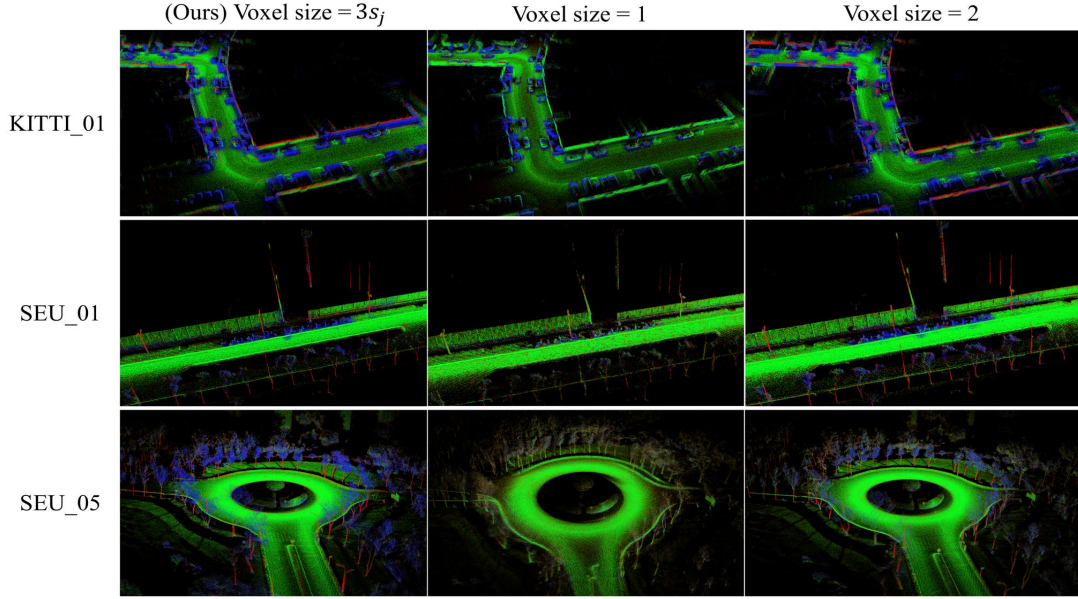


Fig. 6: Results of voxel classification.

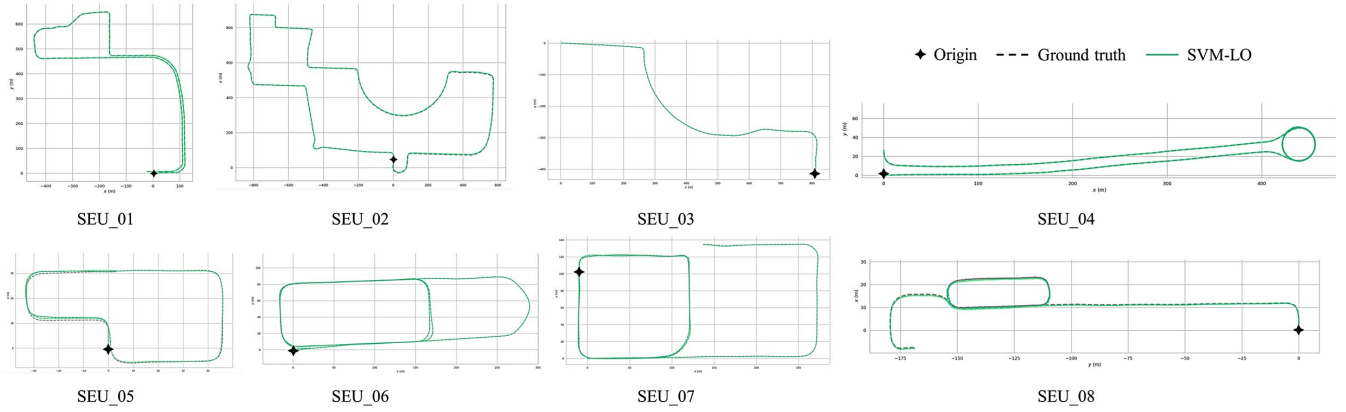


Fig. 7: Trajectory comparison on the SEU dataset.

TABLE III: THE COMPARISON OF ABSOLUTE POSITION ERRORS (APE, METERS) ON OUR DATASET. THE -- INDICATES METHOD FAIL IN THIS SEQUENCE.

Sequence	Length(m)	Duration(s)	SVM-LO (Ours)	KISS-ICP	Livox- mapping	Loam_livox	FAST-LIO2	Point-LIO
SEU_01	2408.830	431.200	1.085	1.121	--	12.807	4.835	10.587
SEU_02	4627.760	879.900	4.835	4.425	--	--	7.789	18.083
SEU_03	1097.910	174.900	1.719	2.693	34.072	--	5.276	4.443
SEU_04	1281.400	240.800	1.183	1.616	--	24.953	3.062	2.621
SEU_05	307.030	135.200	0.611	--	0.651	7.226	0.293	0.670
SEU_06	1715.710	304.100	0.721	1.690	--	17.340	2.548	2.615
SEU_07	1281.470	240.800	0.533	0.613	18.264	--	0.938	1.946
SEU_08	431.120	207.900	0.732	--	2.352	2.913	0.208	0.420
Total	13151.23	2614.800						
Average			1.427	2.026	28.892	13.048	3.119	5.173

TABLE IV: AVERAGE TRANSLATION ERROR (ATE) [%] EVALUATION AND COMPARISON ON THE KITTI DATASET.

Method	Seq. 00	Seq. 01	Seq. 02	Seq. 04	Seq. 05	Seq. 06	Seq. 07	Seq. 08	Seq. 09	Seq. 10	avg
LOAM	0.78	1.43	0.92	0.50	0.57	0.65	0.63	1.12	0.77	0.79	0.816
LeGo_LOAM	1.38	28.03	4.57	1.27	0.91	0.8	0.74	1.40	1.25	1.70	4.205
MULLS	0.54	0.62	0.69	0.35	0.29	0.29	0.27	0.83	0.51	0.61	0.500
SUMA	0.68	1.70	1.20	0.74	0.43	0.54	0.74	1.20	0.62	0.72	0.857
SUMA++	0.64	1.60	1.00	0.67	0.40	0.46	0.34	1.10	0.47	0.66	0.734
LiTAMIN2	0.70	2.10	0.98	1.05	0.45	0.59	0.44	0.95	0.69	0.80	0.875
KISS-ICP	0.53	0.63	0.53	0.36	0.31	0.26	0.33	0.84	0.49	0.57	0.485
SVM-LO(Ours)	0.51	0.61	0.56	0.31	0.25	0.27	0.35	0.82	0.48	0.54	0.470

REFERENCES

- [1] E. Javanmardi, Javanmardi, M., Gu, Y., & Kamijo, S., "Autonomous vehicle self-localization based on probabilistic planar surface map and multi-channel LiDAR in urban area," in *IEEE 20th International Conference on Intelligent Transportation Systems (ITSC)*, 2017, pp. 1-8.
- [2] H. Königshof, Niels Ole Salscheider, and Christoph Stiller, "Realtime 3d object detection for automated driving using stereo vision and semantic information," in *2019 IEEE Intelligent Transportation Systems Conference (ITSC)*, 2019, pp. 1405-1410.
- [3] Y. Zhang, J. Wang, X. Wang, C. Li, and L. Wang, "3D LiDAR-Based Intersection Recognition and Road Boundary Detection Method for Unmanned Ground Vehicle," in *2015 IEEE 18th International Conference on Intelligent Transportation Systems*, 2015, pp. 499-504.
- [4] J. Zhang, and S. Singh, "Low-drift and real-time LiDAR odometry and mapping," *Autonomous Robots*, 2017, 41(2), pp. 401-416.
- [5] T. X. Shan, and B. Englot, "LeGO-LOAM: Lightweight and Ground-Optimized LiDAR Odometry and Mapping on Variable Terrain," *2018 IEEE/Rsj International Conference on Intelligent Robots and Systems (Iros)*, 2018, pp. 4758-4765.
- [6] T. X. Shan, B. Englot, D. Meyers, W. Wang, C. Ratti, and D. Rus, "LIO-SAM: Tightly-coupled LiDAR Inertial Odometry via Smoothing and Mapping," *2020 IEEE/Rsj International Conference on Intelligent Robots and Systems (Iros)*, 2020, pp. 5135-5142.
- [7] P. Dellenbach, J.-E. Deschaud, B. Jacquet, and F. Goulette, "CT-ICP: Real-time Elastic LiDAR Odometry with Loop Closure," in *2022 International Conference on Robotics and Automation (ICRA)*, 2022, pp. 5580-5586.
- [8] J. R. Lin, and F. Zhang, "Loam_livox: A fast, robust, high-precision LiDAR odometry and mapping package for LiDARs of small FoV," *2020 IEEE International Conference on Robotics and Automation (Icra)*, 2020, pp. 3126-3131.
- [9] G. T. Vizzo I, Mersch B, Wiesmann L, Behley J, Stachniss C., "Kiss-icp: In defense of point-to-point icp—simple, accurate, and robust registration if done the right way.," *IEEE Robotics and Automation Letters*, 2023, pp. 1029-36.
- [10] W. Xu, Y. Cai, D. He, J. Lin, and F. Zhang, "FAST-LIO2: Fast Direct LiDAR-Inertial Odometry," *IEEE Transactions on Robotics*, vol. 38, no. 4, pp. 2053-2073, 2022.
- [11] P. Biber, and Wolfgang Straßer., "The normal distributions transform: A new approach to laser scan matching," in *In Proceedings 2003 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS 2003)*, 2003, pp. 2743-2748.
- [12] X. Y. L. Chen, A. Milioto, E. Palazzolo, P. Giguere, J. Behley, and C. Stachniss, "SuMa plus plus : Efficient LiDAR-based Semantic SLAM," *2019 IEEE/Rsj International Conference on Intelligent Robots and Systems (Iros)*, 2019, pp. 4530-4537.
- [13] K.-L. Low, "Linear Least-Squares Optimization for Point-to-Plane ICP Surface Registration," *Chapel Hill, University of North Carolina*, vol. 4, pp. 1-3, 2004.
- [14] J. Serafin, and Giorgio Grisetti., "NlCP: Dense normal based point cloud registration.," in *2015 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2015, pp. 742-749.
- [15] H. Wang, H. Liang, Z. Li, P. Zhou, and L. Chen, "A fast coarse-to-fine point cloud registration based on optical flow for autonomous vehicles," *Applied Intelligence*, vol. 53, no. 16, pp. 19143-19160, 2023.
- [16] P. X. Yue Pan, Yujie He, Zhenlei Shao and Zesong Li, "MULLS: Versatile LiDAR SLAM via Multi-metric Linear Least Square," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*, 2021, pp. 11633-11640.
- [17] J. L. Bentley, "Multidimensional binary search trees used for associative searching," *Communications of the ACM*, vol. 18, pp. 509-517, 1975.
- [18] D. Meagher, "Geometric modeling using octree encoding," *Computer graphics and image processing*, vol. 19, pp. 129-147, 1982.
- [19] W. Xu, and F. Zhang, "FAST-LIO: A Fast, Robust LiDAR-Inertial Odometry Package by Tightly-Coupled Iterated Kalman Filter," *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 3317-3324, 2021.
- [20] C. Yuan, W. Xu, X. Liu, X. Hong, and F. Zhang, "Efficient and Probabilistic Adaptive Voxel Mapping for Accurate Online LiDAR Odometry," *IEEE Robotics and Automation Letters*, vol. 7, no. 3, pp. 8518-8525, 2022.
- [21] X. G. Zhu, H. Zhou, T. Wang, F. Z. Hong, Y. X. Ma, W. Li, H. S. Li, and D. H. Lin, "Cylindrical and Asymmetrical 3D Convolution Networks for LiDAR Segmentation," *2021 IEEE/Cvf Conference on Computer Vision and Pattern Recognition, Cvpr 2021*, pp. 9934-9943, 2021.
- [22] J. Behley, and C. Stachniss, "Efficient Surfel-Based SLAM using 3D Laser Range Data in Urban Environments," *Robotics: Science and Systems Xiv*, 2018.
- [23] L. He, W. Li, Y. Guan, and H. Zhang, "IGICP: Intensity and Geometry Enhanced LiDAR Odometry," *IEEE Transactions on Intelligent Vehicles*, vol. 9, no. 1, pp. 541-554, 2024.
- [24] W. X. Dongjiao He, Nan Chen, Fanze Kong, Chongjian Yuan, Fu Zhang, "Point-LIO: Robust High-Bandwidth Light Detection and Ranging Inertial Odometry," *Adv. Intell. Syst.*, pp. 2200459, 2023.
- [25] K. K. Masashi Yokozuka1, Shuji Oishi1 and Atsuhiko Banno1, "LiTAMIN2: Ultra Light LiDAR-based SLAM using Geometric Approximation applied with KL-Divergence," 2021.
- [26] P. L. a. R. U. A. Geiger, "Are we ready for autonomous driving The KITTI vision benchmark suite," in *2012 IEEE Conference on Computer Vision and Pattern Recognition*, Providence, USA, 2012, pp. 3354-3361.
- [27] J. L. a. F. Zhang, "R3LIVE++: A Robust, Real-time, Radiance reconstruction package with a tightly-coupled LiDAR-Inertial-Visual state Estimator," 2022.